

Appendix A — Security architecture

Orama Network and RootWallet. Status as of October 3, 2026.

Every statement describes code in the repositories today (Orama release branch 1.0.0, RootWallet main). Anything designed but not built is labeled "Planned" or "Not yet". Where code is written but not yet running on the long-lived networks, we say so.

1. Summary

1. **Keys stay on the user's device.** RootWallet is local-only: no account server, no cloud backup. The recovery phrase is the only way back in.
2. **A local program cannot silently spend.** The wallet agent identifies callers from the kernel, pins approval to the caller's binary hash, and requires a person to approve every ORAMA transaction after decoding it.
3. **Nodes talk over an authenticated overlay.** Internal traffic uses WireGuard, and internal endpoints also require a cryptographic stamp, not just a mesh source address.
4. **Every tenant gets its own cluster.** Each namespace has its own database, cache and gateway, with sandboxing that keeps tenant code off the mesh and away from platform tables.
5. **Authority is narrow, short-lived and revocable.** Wallet sign-in, 15-minute tokens, expiring scoped keys, revocation in about ten seconds.
6. **Releases are signed.** Nodes refuse archives that are unsigned, altered, or signed outside the operator's trust anchor.
7. **Secrets are encrypted at rest,** and private files are encrypted before they reach IPFS.
8. **The ledger has no administrator.** The ORAMA chain has no admin key, pause or upgrade authority, and a test checks it.

Not yet true, and covered in Section 7: the locked-down node OS (OramaOS) has never been booted on a live network, Ubuntu node disks are not encrypted, and no external audit has been done.

2. Threat model

Adversary	What we defend
Remote attacker	Authenticated gateway, per-route policy, rate limits, no public listeners other than the proxy
Malicious tenant	Isolation of data, secrets, network reach and processes from other tenants and the platform
Malicious program on the user's machine	Agent approval model, domain-separated signing, clear-signing
Hostile gateway	A server cannot use a login prompt to obtain a build-archive or transaction signature
Compromised release pipeline	Signed archives, node-side trust anchor, TUF checks
Curious node operator	Encrypt-before-store and scoped secrets. Partial; see Section 7
Physical thief	Encryption at rest; PIN lockout on mobile

Out of scope today:

- An operator with root on a standard Ubuntu or Debian node. This is why node operation is invite-only.
- A memory snapshot of a running machine, including by a hosting provider.
- A same-user attacker on the wallet's machine, and password-capturing malware.
- A compromised node as a cluster member: every node holds the cluster secret and is a member of the registry database, so it can still write cluster state.

3. RootWallet

RootWallet has desktop (macOS and Linux; no Windows build), mobile and command-line forms, with an integrated password and SSH-key vault. The desktop app runs a local agent that other tools, including the Orama CLI, call for signatures and secrets.

3.1 Key derivation

One BIP-39 phrase is the root. Chain keys use BIP-32 paths: EVM `m/44'/60'/0'/0'/N`, Solana `m/44'/501'/N'/0'`, Bitcoin Native SegWit `m/84'/0'/N'/c/i`, ORAMA `m/44'/118'/0'/0'/N`. Separate HKDF-SHA256 branches, each with its own salt, give a vault metadata key, a public identity hash, an Orama storage key-wrapping key and an Orama backup key. The backup branch is an X25519 key: a cluster can encrypt backups to its public half and cannot decrypt them. A wallet created from an imported private key has no phrase and no HKDF branches.

3.2 Encryption at rest

The keystore is encrypted with a password-derived key (scrypt, $N=2^{18}$, $r=8$, $p=1$) and AES-256-GCM. Files are 0600 in 0700 directories. Ciphertext is bound to its role through associated data; a reader selects it from the stamped version and never retries another way, so a failed check cannot be turned

into a downgrade. Desktop and mobile add a second password-keyed layer. The Rust agent holds secrets in zeroizing buffers; TypeScript overwrites byte arrays, but strings such as the phrase cannot be zeroed, and the docs say so.

The agent keeps no session file. After unlock it holds the seed and vault key in memory for a 30-minute sliding window and never returns the phrase over its socket.

3.3 The desktop agent trust model

The agent listens on a Unix socket (0600, created under a tightened umask). For each request:

- **Kernel peer credentials.** The caller's process ID comes from `SO_PEERCRED` (Linux) or `LOCAL_PEERPID` (macOS). If identity cannot be established, the request is refused.
- **Start-time binding.** The ID is bound to the process start time, so a recycled ID cannot impersonate an approved tool.
- **Binary-hash approvals.** Capabilities are granted to the SHA-256 of the caller's executable. A rebuilt binary prompts again.
- **UI-only endpoints.** Unlock, lock, the approved-app list and all approval controls are served only to the desktop app's own process, so a local tool cannot approve itself or probe the password.
- **Throttling.** After five wrong passwords, lockouts double from 30 seconds to 15 minutes.
- **Socket ownership.** The agent will not start if it cannot own the socket. The Orama CLI refuses a socket that is a symlink, owned by another user, or group- or world-writable.

Residual risks, documented in the protocol and not closed:

- **Same-user attacker.** A process already running as the user can launch an approved binary or inject code into one (`LD_PRELOAD`, `DYLD_INSERT_LIBRARIES`, `ptrace`), and the request is served. It can also attack the keystore file offline. A separate agent user, an OS sandbox or hardware-backed per-use confirmation would help; none is implemented.
- **Per-tool approval.** Once a binary holds the signing capability, later message signatures are not prompted. ORAMA transactions and archive signatures are exceptions.
- **Same-process bypass.** The app's own process skips the check; an environment variable disables the bypass.
- **No hardware key storage.** The seed is protected by a password-derived key, not a Secure Enclave. On macOS, Touch ID gates retrieval of the wallet password from the login Keychain.

3.4 Domain-separated signing

The same wallet signs gateway logins and Orama build archives, and nodes install archives signed by a trusted wallet. A hostile gateway could therefore return an archive message as its login challenge. The agent reserves formats: an archive message is signed only under an explicit purpose and a separate capability, and plain signing refuses it. Release signing has its own purpose, and the Orama client refuses to send a message under the wrong purpose before opening the socket. Archive approvals show the parsed build and any change to trusted signers, and each signature raises a notification and an audit line.

3.5 Clear-signing of ORAMA transactions

The agent decodes the transaction from the exact bytes it will sign. Every layer must re-encode to those bytes, so a field the agent does not understand is refused instead of being left off the dialog. Before any prompt it refuses an unknown chain identifier, more than one signer, a signer that is not the active account, an unknown message type, extension options, a tip, any fee denomination other than norama, and imported accounts. It decodes bank send, delegate, undelegate, redelegate, create-validator and reward withdrawal, and shows labeled amounts. There is no "sign anyway". Each transaction is approved on its own; approval grants nothing for the next. After approval the agent re-reads the session and signs nothing if the wallet locked or the account changed. One staging chain identifier is accepted today.

A person approves each first grant of a capability, each ORAMA transaction and each archive signature. A person does not approve each login signature or vault read once a grant exists.

3.6 AI agents and the headless agent

To the wallet, an AI coding agent running the Orama CLI is an ordinary local program: it needs a grant and cannot sign an ORAMA transaction without a person approving it. The headless agent for development and CI is compiled only with a cargo feature and absent from release builds. It refuses a real wallet: it needs an isolated home, a marker written only by the test-wallet command, and a check that no test address appears in any real keystore on the machine. It signs transactions only for allowed test chain identifiers with a maximum amount, and only for a binary named exactly as the Orama CLI (a name check; a renamed copy would not be detected).

3.7 Mobile and test vectors

Mobile uses a wallet password, an optional six-digit PIN and optional biometrics. The PIN gates entry only; every action touching key material asks for the password. PIN failures cause exponential lockout (cap 15 minutes) and never wipe data. Mobile has no dApp or message-signing surface.

The core is TypeScript and the desktop agent is Rust, so committed fixtures generated by the TypeScript code (keystores in each envelope version, a vault, script and HKDF outputs, an ORAMA signing vector) must be reproduced by the Rust code. A mismatch in associated data once made every unlock fail, which is why this test exists.

3.8 What was removed

The cloud-backup and guardian-share scheme, its sync engine and the WalletConnect layer were removed in September 2026. Nothing is uploaded. This removes the wallet's network attack surface; the cost is that a lost phrase cannot be recovered by anyone.

4. Orama network

4.1 Network and admission

Inter-node traffic uses a WireGuard overlay (10.0.0.0/24). Firewall rules admit the mesh on the WireGuard interface, so a packet merely claiming an overlay address elsewhere is not admitted. IPv6 is disabled. Database, gateway, pubsub and IPFS listeners bind the overlay or loopback, and local control sockets use Unix sockets with peer-credential checks.

A node joins with an invite token: stored only as a hash, valid at most one hour, consumed atomically, with a uniqueness conflict keeping it spent. The invite carries the SHA-256 fingerprint of the minting node's certificate, which the joiner pins, so there is no trust on first use. Remote installs pass it on standard input, never a command line.

Each node has an Ed25519 key for self-registration and heartbeat calls, enrolled with proof from its libp2p identity; revoking a node disables its key at once. This is partial. Node-to-node coordination calls (namespace spawn, secret re-encryption, replica operations) use a MAC derived from the shared cluster secret, which proves cluster membership and not which node signed. The stamp covers method, path, query, body hash, nonce, a one-minute window and the receiving node.

4.2 Authentication and authorization

Sign-in is Sign-In with Ethereum (EIP-4361) or its Solana equivalent. The signed message names the gateway's host, the namespace, a single-use nonce and a five-minute expiry, and the gateway reads those from the signed text. The result is a 15-minute access token and a rotating refresh token stored as a hash. Each gateway signs with its own Ed25519 key, and a namespace gateway's key is bound to its namespace. Keys rotate without logging anyone out.

A replicated revocation list, reloaded every five seconds, invalidates revoked keys and ended sessions (including open WebSockets) in about ten seconds. Sessions can be bound to a device key.

API keys carry a checksum that secret scanners can match, hide their namespace, are stored as HMACs, and expire (90 days by default, one year at most). Rotation keeps the old key valid for seven days. Permissions take the form domain:action:resource. Roles are owner (one per namespace, enforced by a unique index), admin, developer, runtime and reader, and a grant can be narrowed to pubsub topics, one function, a storage prefix or cache keys.

Every route declares an access policy. A route registered without one stops the process from starting, and tests fail on contradictory policies. An unmatched path requires a credential and reaches nothing. Sign-ins, key and grant changes, deployments, secret changes and operator actions go to a replicated audit trail kept 90 days; secret values and credentials are never recorded. Gateway-to-gateway calls carry an HMAC over method, path and asserted identity; the first middleware deletes internal headers lacking a valid MAC, and Caddy strips them from public traffic.

4.3 Tenant isolation

- **Own cluster.** Each namespace has its own RQLite, Olric cache and gateway.

- **SQL guard.** Tenant SQL naming any of more than 50 platform tables, in any position, is refused, as are ATTACH and PRAGMA. The code states this is a filter, not the final fix; moving platform state out of any database a tenant can name is the direction.
- **Tenant apps.** Each deployment runs as a dynamically allocated unprivileged user, sees only its own directory, binds only its own port, cannot reach private ranges, and has memory, CPU and task limits. Node.js dependency installs run in a separate sandboxed unit and accept registry packages only.
- **Daemons.** Services run under systemd sandboxing (read-only system, no new privileges, restricted namespaces, hidden processes, no swap). Root actions go through a socket-activated helper that checks the caller's kernel-reported identity and systemd unit; there is no sudo. Namespace daemons still share one operating-system account; separation is by sandboxing and path binding. Only DNS and the media server have their own accounts.
- **Serverless.** Functions run in WebAssembly (wazero) with a memory cap (default 64 MB, at most 256 MB) and a time limit (default 30 seconds, at most five minutes). Outbound HTTP is checked on the socket at connect time, for every resolved address and every redirect hop, against reserved ranges (loopback, private, link-local, carrier-grade NAT, multicast, and IPv6 forms wrapping IPv4). A function reaches only its own namespace's database and runs only in its own namespace's gateway.

4.4 Secrets, storage, calls

TURN, function, push and agent secrets and deployment environment variables are encrypted with AES-256-GCM under a versioned key hierarchy, and an operator command re-encrypts them under a new root. Rotating the cluster secret itself is a maintenance window, not a command. Private uploads, WebAssembly modules and database backups are sealed with AES-256-GCM before being added to IPFS; directories and tarball deployments are not. Deleted rows persist in the replication log until compaction.

Calls use a media server that forces relay-only ICE, so participants do not learn each other's addresses. Stealth TURN serves TURN over TLS on port 443 behind an SNI router, using a hashed hostname. TURN credentials are namespace-wide, last 24 hours and cannot be revoked per user. Outbound anonymity uses a Tor client in a hardened unit that cannot reach the overlay; there is no direct path, and Tor refuses internal addresses.

4.5 Releases

Archives are signed through the wallet agent. Each node holds a trust anchor, a root-owned file of allowed signer addresses outside any archive-writable directory. There is no built-in signer: a cluster trusts its operator. Push, install and upgrade verify the manifest signature, every file hash, the absence of extra files and links, and the architecture, with no unsigned mode. Signer rotation is supported and guarded against replay of old builds. TUF checks (threshold, timestamp expiry, snapshot rollback, target hashes) are implemented for archive and chain-binary staging with an end-to-end test. A signature proves who built an archive, not that it is the newest, so rollback to an older signed build is allowed on the wallet path. Automatic node updates exist as a library that nothing calls yet.

4.6 Disk encryption and the vault

- **OramaOS.** No shell or SSH, read-only root, an encrypted data partition whose key is Shamir-split across peers, and signed A/B updates. Built, with an emulator first-boot test, but never booted on a live network. Integrity verification is in the image but not wired into boot, the syscall filter audits and does not block, the first node's key has no escrow, and the update path has defects.
- **Ubuntu nodes.** Disks are not encrypted. A design exists and is marked design-only.
- **Vault.** A Zig guardian service splits secrets with Shamir sharing. Guardian discovery and resharing are not finished, and we do not recommend it as the only copy of anything.
- **Validator keys.** Export is sealed to an operator key and guarded against double signing, with limits stated in the docs.

5. ORAMA chain (summary; see Appendix B)

The chain is implemented and tested. It is not what a node install starts, and it does not carry value.

- **No admin keys.** Standard modules that expect a governance authority are bound to an address no key can sign for; a test checks that each authority-gated message is rejected. Changing this takes a coordinated hard fork.
- **Capped voting power.** Power uses capped stake shares (5 percent, 3 percent for a larger validator set) after a 30-epoch ramp, blended with an equal bootstrap-committee share.
- **Inclusion lists.** Transactions seen by two thirds of voting power cannot be censored by a proposer; this is switched on by a genesis parameter.
- **Shielded pool.** User-to-user payments are shielded, reusing the Zcash Orchard verifier through a Rust library. A bundle needs two verifiers to accept: the linked library and a separately built, pinned binary. Both use the same upstream crate, so a logic bug there would be shared. An independent implementation is an open item.
- **Invariants.** Fund-holding modules check their own books (supply against schedule, escrow, pool balances) on every node and at genesis.
- **Declared limit.** A contract can issue a public token against ORAMA it holds. The chain does not prevent it; the docs declare it.

No external audit of the chain has been done.

6. Engineering process

- **Self-audits.** An earlier review that treated the hosting provider as the adversary produced 90 findings (whitepaper figure). Two September 2026 audits covered stability (35 tracked items, all implemented and in review) and authentication (29 items: 26 in review, two in progress, one not started). "In review" means implemented and awaiting a second pass. RootWallet had a three-round audit on July 6, 2026; its file marks which findings were fixed or removed with their code, and unmarked findings are not re-verified, so we do not count them closed.

- **Tests.** Orama core has about 7,700 Go test functions, the chain about 1,260, the Zig vault 196 tests. Tests walk the source to enforce rules, such as every 401 or 403 carrying a code and every route having a policy.
- **Fleet end-to-end suite.** 81 feature packages across eleven stages, including chaos and soak packages and release, firewall, invite-join and security-audit packages. The most recent recorded run (October 2, 2026, five-node staging network) passed 480 of 497 recorded tests with one failure and several features without a result, so its verdict was FAIL. We have no fully green run yet.
- **Documentation claims.** A test checks documented claims against the code and a live fleet and names the file and line to correct.
- **Rollout.** The hardening is on branch 1.0.0 and has run on the staging network. Rollout to the long-lived development and test networks is pending.
- **Disclosure.** A disclosure policy and severity scale exist. There is no paid bounty: no amounts and no funding source.

7. Known gaps and what this round funds

- No external audit of RootWallet, the Orama services, the chain or the zero-knowledge integration.
- OramaOS is not proven, so untrusted operators cannot safely run nodes.
- Ubuntu disks are unencrypted, memory is exposed to the host, and a disk snapshot exposes the cluster's shared secrets.
- Every node holds the cluster secret and is a registry member. Rotating that secret and the WireGuard keys needs a maintenance window; no mesh key rotation tooling exists.
- During mixed-version rollout, an older coordination stamp is still accepted on a few read-only routes. Removal is scheduled for the next release.
- Some mesh services (media signaling, message gossip) do not authenticate each peer individually.
- Tenant daemons share an operating-system account, and a deployed app can dial tenant-reachable loopback ports.
- Wallet: same-user attacker, no hardware-backed key, no Windows build, per-tool signing approval.
- Both shielded-pool verifiers share one upstream crate.

This round funds:

- an external audit of RootWallet;
- in-house audits of the Orama services and the zero-knowledge integration by the cryptography and security engineers we are hiring;
- finishing OramaOS (boot integrity, enforcing syscall filters, update path, key escrow);
- rotation tooling for the cluster secret and mesh keys;
- an independent second verifier for the shielded pool;
- confining each node to its own registry rows, which is a topology change;
- a paid bounty program after an audit.